

Session 9: Deadlock

1. Necessary Conditions for Deadlock

A **deadlock** occurs when processes cannot proceed because each is waiting for a resource held by another. For a deadlock to occur, four conditions must hold simultaneously:

1. **Mutual Exclusion:**

- At least one resource is non-shareable (can only be used by one process at a time).
- Example: A printer can be used by only one process at a time.

2. **Hold and Wait:**

- A process holding one resource is waiting for additional resources held by other processes.
- Example: Process A holds a file and waits for a printer, while Process B holds the printer and waits for the file.

3. **No Preemption:**

- Resources cannot be forcibly taken from a process; they must be released voluntarily.
- Example: If Process A has allocated memory, the OS cannot reclaim it until A releases it.

4. **Circular Wait:**

- A set of processes are waiting in a circular chain, where each process waits for a resource held by the next process.
- Example: Process A waits for Process B's resource, Process B waits for Process C's resource, and Process C waits for Process A's resource.

2. Deadlock Prevention and Avoidance

2.1 Deadlock Prevention

Prevention strategies ensure that at least one of the necessary conditions for deadlock does not occur.

- **Mutual Exclusion:**

- Make resources shareable where possible.
- Example: Use spoolers for printers to allow multiple jobs to queue.

- **Hold and Wait:**

- Require processes to request all required resources at once.
- **Disadvantage:** Can lead to resource underutilization.

- **No Preemption:**

- Allow the OS to forcibly take resources from a process.
- Example: A process holding memory can have it swapped out.

- **Circular Wait:**

- Impose a strict order of resource allocation.
 - Example: If Process A requests Resource 1, it must request Resource 2 in ascending order.
-

2.2 Deadlock Avoidance

Avoidance strategies dynamically check whether allocating a resource will lead to a deadlock.

- **Banker's Algorithm:**
 - Evaluates resource allocation requests based on the system's current state and ensures safe resource allocation.
 - Requires:
 1. Maximum resource requirement of each process.
 2. Currently allocated resources.
 3. Available resources.
 - Example:
 1. Total Resources: 10.
 2. Available Resources: 3.
 3. Process P1 needs 2 more resources; allocation is safe as enough resources will remain for others.
-

3. Semaphore

A **semaphore** is a synchronization tool used to manage concurrent processes by controlling access to shared resources.

- **Types:**
 1. **Binary Semaphore:** Takes values 0 or 1; works like a mutex.
 2. **Counting Semaphore:** Tracks the number of available resources.
- **Operations:**
 1. **Wait (P):** Decreases the semaphore value. If it becomes negative, the process is blocked.
 2. **Signal (V):** Increases the semaphore value. If a process is waiting, it is unblocked.

Example:

- A semaphore initialized to 1 controls access to a file.
 - **Process A** performs a **Wait**, reducing the semaphore to 0.
 - **Process B** attempting **Wait** is blocked until A performs **Signal**.
-

4. Mutex

A **mutex** (mutual exclusion) is a locking mechanism that ensures only one process accesses a critical section at a time.

- **Difference from Semaphore:**
 - Mutex can only be unlocked by the thread that locked it, whereas a semaphore can be signaled by any thread.
 - Example:
 - Mutex ensures exclusive access to a database update operation.
-

5. Producer-Consumer Problem

The **producer-consumer problem** demonstrates the use of semaphores for synchronization between two processes.

- **Problem:**
 - A **producer** generates data and places it in a buffer.
 - A **consumer** retrieves data from the buffer.
 - Synchronization is needed to avoid:
 - Buffer overflow (producer adds when full).
 - Buffer underflow (consumer removes when empty).
 - **Solution Using Semaphores:**
 - **Semaphores:**
 - `empty`: Tracks empty slots in the buffer.
 - `full`: Tracks filled slots.
 - `mutex`: Ensures mutual exclusion.
 - **Algorithm:**
 - Producer:

```
wait(empty);
wait(mutex);
Add item to buffer;
signal(mutex);
signal(full);
```
 - Consumer:

```
wait(full);
wait(mutex);
Remove item from buffer;
signal(mutex);
signal(empty);
```
-

6. Deadlock vs. Starvation

Aspect	Deadlock	Starvation
Definition	All processes are blocked indefinitely, waiting for resources.	A process waits indefinitely due to resource prioritization.
Cause	Circular wait and resource dependency.	Lower-priority processes are bypassed repeatedly.
Resolution	Requires breaking the deadlock (e.g., killing processes).	Adjust scheduling policies (e.g., aging).
Example	Four processes hold and wait for resources in a circular chain.	A low-priority process in a priority queue waits indefinitely.

This detailed explanation covers **deadlock**, its conditions, prevention and avoidance strategies, synchronization tools like **semaphores** and **mutexes**, and common problems like the **producer-consumer problem**. It also highlights the distinction between **deadlock** and **starvation** with examples for clarity.